

「Dependent スコープのインスタンスがメモリ解放されない」という現象が、CDI の仕様上なぜ起きてしまうのでしょうか。

それでは、エンジニアの間でも「ハマりどころ」として有名な、この **Dependent スコープのメモリリーク問題**について、CDI の仕様の観点から解説します。

結論から言うと、これはバグではなく**「CDI の仕様通りの挙動」**であるところが厄介なポイントです。

なぜメモリが解放されないのか？

原因は、CDI 仕様における**「Dependent オブジェクトのライフサイクル管理」**のルールにあります。

1. ライフサイクルが「注入先」に依存する

@Dependent スコープのインスタンスは、名前の通り「依存（Dependent）している側」のオブジェクトと同じ寿命を持ちます。

- **通常のインジェクションの場合:** RequestScoped な Bean に注入されれば、リクエスト終了時に一緒に破棄されます。
- **動的ルックアップ (Instance<T>.get()) の場合:** プログラム中で CDI.current().select(...).get() と呼び出した場合、そのインスタンスは「ルックアップを実行したコンテキスト (CreationalContext)」に紐付けられます。

2. 親が死ぬまで解放されない

動的に取得したインスタンスは、CDI コンテナ側から見ると**「誰かが明示的に破棄（destroy）しない限り、親（取得元）が消えるまで保持し続けなければならないもの」**として扱われます。

特に、ApplicationScoped なクラス内や、静的なメソッドから動的に Dependent な Bean を取得し続けると、その「親」がアプリケーション終了まで生存するため、取得したインスタンスへの参照が内部の CreationalContext に溜まり続け、GC（ガベージコレクション）の対象にならなくなります。

仕様上の定義

CDI 仕様書では、`Instance<T>.get()` で取得したインスタンスについて以下のように定めています。

Instance オブジェクトを介して取得した Dependent スコープの Bean インスタンスは、その Instance オブジェクトが破棄されるか、または `Instance.destroy()` メソッドによって明示的に破棄されるまで、メモリに保持される。

つまり、**「使ったら自分で掃除（destroy）してね」**というのが CDI のスタンスなのです。

どうやって解決するのか？

この問題を回避するには、以下のいずれかの対策をとるのが一般的です。

1. **明示的に破棄する:** `Instance<T>` を使って取得した場合は、使い終わった後に

`instance.destroy(producedObject)` を呼び出します。

2. **スコープを見直す:** 動的に何度も生成・破棄されるべきものは、`@Dependent` ではなく

`@RequestScoped` など、コンテナが自動で掃除してくれるスコープに変更します。

3. **Provider インターフェースを利用する:** 必要以上に参照を保持しないよう、設計を見直します。

この「Dependent の罠」は、特に WildFly (Weld) などの標準実装で顕著に出るため、大規模なアプリケーションでは致命的な `OutOfMemoryError` の原因になりがちです。